STADTFEST

Die Stadtfest-Kasse

Akte 3 · DS 5 „Kassensturz“ · DS 6 „Die Statistik lügt nicht“

Kassensturz

BRIEFING · EDDA LORENZ

Frauke Melchers, Kassenwartin des Festausschusses, hat uns alle Kassendaten des Stadtfests gebracht – und eine Zahl: 312,50 € Fehlbetrag. Die Datenbank ist groß: Stände, Helferinnen und Helfer, Schichten, Produkte, Bons und Positionen. Einzelne Zeilen anzuschauen bringt hier nichts; wir müssen verdichten – zählen, summieren, Durchschnitte bilden. Und es gibt eine eiserne Regel: Jede Spalte im SELECT ist entweder aggregiert oder steht im GROUP BY. Gleich sortiert ihr euch selbst in Fächer: Ihr seid die Datensätze.

Die Datenbank Stadtfest

Stand(StandNr, Name, Art, Platz)

Helfer(HelferNr, Vorname, Nachname, Jahrgang, Verein)

Schicht(↑HelferNr, ↑StandNr, Beginn, Ende)

Produkt(ProduktNr, Bezeichnung, Preis, ↑StandNr)

Bon(BonNr, ↑StandNr, ↑HelferNr, Zeitpunkt, Zahlart)

Position(↑BonNr, ↑ProduktNr, Menge)

Ein **Bon** hat mehrere **Positionen**; jede Position nennt ein Produkt und eine **Menge**.

Impuls: Zählen oder summieren?

- Wie viele Bons gab es am Stadtfest?
- Wie viele Bratwürste wurden verkauft?
- Ist das dieselbe Art von Zahl?
- Wir wollen die Anzahl der Bons **je Stand**. Wie viele Ergebniszeilen erwartet ihr?

Aggregatfunktionen

Fachbegriff	Bedeutung
`COUNT(*)`	Anzahl der Zeilen
`COUNT(Spalte)`	Anzahl der Werte ungleich NULL
`SUM(Spalte)`	Summe, z. B. verkaufte Stück `SUM(Menge)`
`AVG(Spalte)`	Durchschnitt
`MIN` / `MAX`	kleinster / größter Wert

Ohne GROUP BY verdichtet eine Aggregatfunktion die ganze Tabelle zu **einer** Ergebniszeile.

Kennzahlen ohne GROUP BY

```
SELECT COUNT(*) AS Produkte, MIN(Preis) AS Billigste,
       MAX(Preis) AS Teuerste, AVG(Preis) AS Schnitt
FROM Produkt;
```

ERGEBNIS

Produkte	Billigste	Teuerste	Schnitt
26	1	6,5	3,35

1. **FROM** – alle 26 Zeilen der Tabelle Produkt
2. **SELECT** – jede Aggregatfunktion verdichtet die Spalte Preis zu einem Wert
3. Ergebnis: **eine** Zeile

GROUP BY bildet Fächer

1 · ALLE BON-KARTEN (FROM Bon)

Bon 101	Bon 102	Bon 104	Bon 105	Bon 106	Bon 108	Bon 109	Bon 110	Bon 113
Menge 1	Menge 2	Menge 4	Menge 1	Menge 1	Menge 3	Menge 1	Menge 3	Menge 2
Kaffeemobil	Grillhütte	Crêpes-Stand	Kaffeemobil	Grillhütte	Crêpes-Stand	Kaffeemobil	Grillhütte	Kaffeemobil

2 · GROUP BY Stand – je Stand ein Fach

FACH KAFFEEMOBIL

Bon 101	Bon 105	Bon 109	Bon 113
Menge 1	Menge 1	Menge 1	Menge 2

`COUNT(*) = 4` Zähler
`SUM(Menge) = 5` Summierer

FACH GRILLHÜTTE

Bon 102	Bon 106	Bon 110
Menge 2	Menge 1	Menge 3

`COUNT(*) = 3` Zähler
`SUM(Menge) = 6` Summierer

FACH CRÊPES-STAND

Bon 104	Bon 108
Menge 4	Menge 3

`COUNT(*) = 2` Zähler
`SUM(Menge) = 7` Summierer

3 · JE FACH GENAU EINE ERGEBNISZEILE

Stand	COUNT(*)	SUM(Menge)
Kaffeemobil	4	5
Grillhütte	3	6
Crêpes-Stand	2	7

Meiste Bons:
Kaffeemobil (COUNT)

Meiste Stück:
Crêpes-Stand (SUM)

Beispiel: Sortiment je Stand

```
SELECT Stand.StandNr, Stand.Name, COUNT(*) AS Produkte,
       MAX(Produkt.Preis) AS Hoechstpreis
FROM Stand, Produkt
WHERE Stand.StandNr = Produkt.StandNr
GROUP BY Stand.StandNr, Stand.Name
ORDER BY Produkte DESC, Stand.StandNr;
```

ERGEBNIS

StandNr	Name	Produkte	Hoechstpreis
4	Kaffeemobil	6	3,5
1	Grillhütte	4	4,5
3	Saftbar	4	4,5
…	…	…	…

1. **FROM/WHERE** – Stand und Produkt verbinden
2. **GROUP BY** – ein Fach je Stand (Schlüssel + Name)
3. **SELECT** – je Fach Anzahl und Höchstpreis
4. **ORDER BY** – größtes Sortiment zuerst

Menschliches GROUP BY

UNPLUGGED · 15 MIN · MATERIAL: 30 BON-KARTEN, 4 FÄCHERSCHILDER

1. Jede/r bekommt eine Bon-Karte. `SELECT COUNT(*) FROM Bon`: alle stehen auf, eine Person zählt – eine Zahl.
2. `GROUP BY Stand`: alle gehen in die Ecke ihres Standes. Je Fach ein **Zähler** (COUNT) und ein **Summierer** (SUM Betrag) – Ergebnis an die Tafel.
3. `SELECT Stand, Helfer, COUNT(*) … GROUP BY Stand`: Welchen Helfer meldet euer Fach? → GROUP-BY-Regel.
4. `SUM(Menge)` statt `COUNT(*)`: Welches Fach gewinnt jetzt?
5. `WHERE Zahlart = 'Karte'`: Barzahler setzen sich **vor** dem Gruppieren.

Die GROUP-BY-Regel

Jede Spalte im SELECT ist entweder aggregiert oder steht in GROUP BY. Diese Abfrage läuft in SQLite – trotzdem ist sie falsch:

```
SELECT Stand.Name, Produkt.Bezeichnung, COUNT(*) AS Produkte
FROM Stand, Produkt
WHERE Stand.StandNr = Produkt.StandNr
GROUP BY Stand.StandNr, Stand.Name;
```

Welche Bezeichnung soll das Fach Kaffeemobil melden? Es hat sechs. Und: **nach dem Schlüssel** gruppieren – am Stadtfest halfen zwei Personen mit genau demselben Namen.

Ab ins Lab: Ermittlung 3.1

SPUR IM LAB

„Kassensturz“

Aufgaben: Einstieg A3-E1-04 (Sortierfächer), dann A3-E1-01 … 08

Werkzeug: Sortierfächer

- AB 3.1 parallel bearbeiten (Aufträge 1–5, Abi-Ecke).
- Schnelle: Cold Case „Der stille Stand“ im Lab, „Kartenzahler“ auf dem AB.

Ermittler-Notiz: Aggregation

ERMITTLER-NOTIZ

- Ohne GROUP BY: genau **eine** Ergebniszeile.
- GROUP BY bildet Fächer – je Fach **eine** Zeile.
- **GROUP-BY-Regel:** jede SELECT-Spalte aggregiert oder in GROUP BY.
- Nach dem **Schlüssel** gruppieren, Namen zusätzlich aufnehmen.
- „Anzahl Bons“ = `COUNT(*)` · „verkaufte Stück“ = `SUM(Menge)` · `COUNT(Spalte)` übersieht NULL.

Abi-Format

ABI-ECKE · OPERATOR ENTWERFEN

Gegeben ist die Datenbank Stadtfest. **Entwerfen** Sie eine SQL-Abfrage, die für jeden Stand die Standnummer, den Namen und die Anzahl der dort verkauften Produkte (Stückzahl) ausgibt. [4 BE]

Fallbesprechung

BRIEFING · EDDA LORENZ

Am Kaffeemobil gibt es Bons voller Positionen – aber auffällig viele davon mit der Menge 0. Wie finden wir nur die auffälligen Gruppen?



Die Statistik lügt nicht

Akte 3 · DS 6 · Ermittlung 3.2

Die Statistik lügt nicht

BRIEFING · EDDA LORENZ

Die Bons mit Menge 0 lassen mir keine Ruhe. Stellt euch vor: Jemand bekommt einen Kaffee, zahlt bar, und die Kasse bucht nichts – so verschwindet Geld, ohne dass die Kasse falsch rechnet. Wir müssen wissen, wann und bei wem: Dazu filtern wir Gruppen mit HAVING. Vorsicht beim Verbinden: Wer Bons, Positionen und Schichten in einen Topf wirft, kann Zeilen vervielfachen – dann lügt die Statistik eben doch. Und weil im Abitur gern eine fertige Abfrage vorgelegt wird, analysiert ihr heute auch Abfragen im Sachzusammenhang.

Impulse

- Wie zeigen wir nur Personen mit **mehr als 3** Positionen der Menge 0? Kann das in WHERE stehen?
- Juri hat Bons, Positionen und Schichten verbunden – plötzlich ist der Fehlbetrag **dreimal so hoch**. Was ist passiert?

Die Zeitlupe: Auswertungsreihenfolge

Fachbegriff	Bedeutung
1 FROM	Tabellen holen und verbinden
2 WHERE	Zeilen filtern (vor dem Gruppieren)
3 GROUP BY	Fächer bilden
4 HAVING	Fächer filtern (nach dem Gruppieren)
5 SELECT	Spalten und Aggregate berechnen, Aliase vergeben
6 ORDER BY	sortieren (Alias erlaubt)
7 LIMIT	nach n Zeilen abschneiden

WHERE und HAVING in einer Abfrage

```sql
SELECT Produkt.Bezeichnung, SUM(Position.Menge) AS Stueck
FROM Bon, Position, Produkt
WHERE Bon.BonNr = Position.BonNr
  AND Position.ProduktNr = Produkt.ProduktNr
  AND Bon.Zeitpunkt LIKE '2026-08-28%'
GROUP BY Produkt.ProduktNr, Produkt.Bezeichnung
HAVING SUM(Position.Menge) >= 40
ORDER BY Stueck DESC;
```

ERGEBNIS

Bezeichnung	Stueck
Los	67
Alkoholfreies Bier	47

1. **WHERE** – nur Bons vom Freitag (Zeilenbedingung)
2. **GROUP BY** – ein Fach je Produkt
3. **HAVING** – nur Fächer mit mindestens 40 Stück
4. **SELECT/ORDER BY** – Bezeichnung und Stück, meistverkauft zuerst

Menschliche Zeitlupe

UNPLUGGED · 10 MIN · MATERIAL: 7 KLAUSELKARTEN, 12 BEDINGUNGSKARTEN

1. Sieben Freiwillige halten die Klauselkarten in **Schreibreihenfolge** hoch.
2. Das Plenum sortiert sie in **Auswertungsreihenfolge** um. Impuls: Warum kann WHERE noch nicht wissen, wie viele Bons eine Gruppe hat?
3. Teams sortieren 12 Bedingungskarten auf zwei Stapel: „vor dem Gruppieren (WHERE)“ / „nach dem Gruppieren (HAVING)“ – Wettlauf.
4. Auflösung durch die Klausel-Personen; Zusatzfrage: Alias in ORDER BY ja, in HAVING nein – warum?

Top-n mit LIMIT – Vorsicht, Gleichstand

```sql
SELECT Helfer.HelferNr, Helfer.Vorname, Helfer.Nachname,
       COUNT(*) AS Schichten
FROM Helfer, Schicht
WHERE Helfer.HelferNr = Schicht.HelferNr
GROUP BY Helfer.HelferNr, Helfer.Vorname, Helfer.Nachname
ORDER BY Schichten DESC, Helfer.HelferNr;
```

ERGEBNIS

HelferNr	Vorname	Nachname	Schichten
33	Enno	Kolthoff	4
35	Arne	Thölken	4
40	Mats	Vosgerau	4
51	Ida	Rohlfs	4
1	Pauline	Hedden	3
…	…	…	…

1. `ORDER BY … DESC LIMIT 3` – die „drei fleißigsten“ Personen?
2. Vier Personen haben je 4 Schichten.
3. `LIMIT 3` schneidet eine davon willkürlich ab.
4. Besser: „alle mit mindestens 4 Schichten“ (`HAVING COUNT(*) >= 4`)

Vervielfachung im Verbund

Bon 486 hat **eine** Position (Kaffee, Menge 0). Wird Schicht nur über den Stand verbunden, passt jede Schicht des Kaffeemobils ab 21 Uhr – aus einer Position werden **drei Zeilen**:

```
SELECT Bon.BonNr, Position.ProduktNr, Schicht.HelferNr, Schicht.Beginn
FROM Bon, Position, Schicht
WHERE Bon.BonNr = Position.BonNr
  AND Bon.StandNr = Schicht.StandNr
  AND Bon.BonNr = 486
  AND Schicht.Beginn >= '2026-08-29 21:00'
ORDER BY Schicht.Beginn;
```

BonNr	ProduktNr	HelferNr	Beginn
486	11	7	2026-08-29 21:00
486	11	23	2026-08-29 22:00
486	11	7	2026-08-29 23:00

Abfragen analysieren und Vorgehen beschreiben

Analyse im Sachzusammenhang: „Die Abfrage liefert für jede/jeden … die/den …, berücksichtigt werden nur …, ausgegeben werden nur Gruppen, die …, sortiert nach …“ – nicht: „wählt aus und gruppiert“.

Vorgehen in Worten:

1. Tabellen
2. Verbund
3. Zeilenfilter (WHERE)
4. Gruppierung
5. Gruppenfilter (HAVING)
6. Sortierung/Begrenzung

Ab ins Lab: Ermittlung 3.2

SPUR IM LAB

„Die Statistik lügt nicht“

Aufgaben: A3-E2-01 … 08 · Cold Case A3-E2-CC

Werkzeug: Zeitlupe

- 06 und 08: Analyse in eigenen Worten – die Antwort bleibt auf deinem Gerät.
- Danach: Verhör 3 · Schulkiosk (20 min, gesiezt, wie im Abitur).

Ermittler-Notiz: WHERE, HAVING, LIMIT

ERMITTLER-NOTIZ

- FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT
- WHERE filtert **Zeilen vorher**, HAVING filtert **Fächer nachher** – Aggregat wiederholen, kein Alias.
- Top-n mit LIMIT nur eindeutig ohne Gleichstand an Platz n.
- Vervielfachung: nur Tabellen verbinden, die die Frage braucht.
- Analyse: Information im Sachzusammenhang, nicht Syntax nacherzählen.

Abi-Format

ABI-ECKE · OPERATOR ANALYSIEREN

Analysieren Sie die Abfrage, indem Sie beschreiben, welche Informationen sie im Sachzusammenhang liefert. [4 BE]

```
SELECT Produkt.Bezeichnung, SUM(Position.Menge) AS Stueck
FROM Bon, Produkt, Position
WHERE Bon.BonNr = Position.BonNr
  AND Position.ProduktNr = Produkt.ProduktNr
  AND Bon.Zeitpunkt LIKE '2026-08-29%'
GROUP BY Produkt.ProduktNr, Produkt.Bezeichnung
HAVING SUM(Position.Menge) > 50
ORDER BY Stueck DESC;
```